

Dataset Link: [Microsoft Adventure Works Cycles Customer data](#)

PART 1

Based on Feature Selection, Cleaning, and Preprocessing to Construct an Input from Data Source

- (a) Examine the values of each attribute and Select a set of attributes only that would affect to predict future bike buyers to create your input for data mining algorithms. Remove all the unnecessary attributes. (Select features just by analysis).

SOLUTION

1. Inspecting Data

```
import pandas as pd
import numpy as np
```

```
df = pd.read_csv('./AWCustomers.csv')
print(df.columns)
```

OUTPUT

```
Index(['CustomerID', 'Title', 'FirstName', 'MiddleName', 'LastName', 'Suffix',
      'AddressLine1', 'AddressLine2', 'City', 'StateProvinceName',
      'CountryRegionName', 'PostalCode', 'PhoneNumber', 'BirthDate',
      'Education', 'Occupation', 'Gender', 'MaritalStatus', 'HomeOwnerFlag',
      'NumberCarsOwned', 'NumberChildrenAtHome', 'TotalChildren',
      'YearlyIncome', 'LastUpdated'],
      dtype='object', name='columns')
```

2. Selecting Relevant Attributes

- (a) **Age:** Age strongly influences lifestyle, mobility needs, and health goals.
- (b) **Education:** Education level often correlates with awareness of health, environmental issues, and sustainable transport benefits.
- (c) **Occupation:** Work type affects commute distance, income, and time availability.
- (d) **Gender:** Gender can reflect differences in purchasing motivations, preferred cycle types, and
- (e) **MaritalStatus:** Family structure affects spending priorities and intended use.
- (f) **HomeOwnerFlag:** Homeownership can be a proxy for financial stability and storage availability.
- (g) **NumberCarsOwned:** Indicates transport preferences and potential openness to cycling.
- (h) **YearlyIncome:** Income determines affordability and the likelihood of purchasing premium or multiple bicycles.

∴ selecting only these attributes for the input DataFrame.

PART 1

Based on Feature Selection, Cleaning, and Preprocessing to Construct an Input from Data Source

- (b) Create a new Data Frame with the selected attributes only.

SOLUTION

```
selected_features = [
    'age',
```

```

    'education',
    'occupation',
    'gender',
    'maritalstatus',
    'homeownerflag',
    'numbercarsowned',
    'yearlyincome'
]

df_selected = df[selected_features].copy()

```

PART 1

Based on Feature Selection, Cleaning, and Preprocessing to Construct an Input from Data Source

- (c) Determine a Data value type (Discrete, or Continuous, then Nominal, Ordinal, Interval, Ratio) of each attribute in your selection to identify preprocessing tasks to create input for your data mining.

SOLUTION

- (a) **Age:** Continuous (Ratio) because it is measured in years with a true zero and equal intervals, allowing meaningful comparisons. Age reflects lifestyle stage, fitness levels, and transport needs, influencing whether cycling is for commuting, sport, or leisure.
- (b) **Education:** Discrete (Ordinal) because it consists of ordered categories (e.g., high school, bachelor's, master's) with no fixed interval between them. Education level often correlates with health awareness and environmental consciousness, affecting likelihood of bicycle purchases.
- (c) **Occupation:** Discrete (Nominal) because it categorizes individuals without intrinsic order (e.g., engineer, teacher, manager). Work type impacts commute habits, income level, and time available for cycling.
- (d) **Gender:** Discrete (Nominal) because it is a categorical variable with no inherent order. Gender may influence bicycle preferences, design choices, and responsiveness to marketing.
- (e) **MaritalStatus:** Discrete (Nominal) because it categorizes without ranking (e.g., single, married, divorced). Household structure affects spending priorities and whether purchases are for individuals or families.
- (f) **HomeOwnerFlag:** Discrete (Binary) because it has only two possible values (yes/no). Homeownership can indicate financial stability and the availability of space for bicycle storage.
- (g) **NumberCarsOwned:** Discrete (Ratio) because it has a true zero and allows ratio comparisons. It serves as a proxy for transportation reliance—fewer cars may indicate cycles are used for commuting, more cars for recreation.
- (h) **YearlyIncome:** Continuous (Ratio) because it has a true zero and equal intervals, allowing meaningful ratios. Income directly affects affordability and the type of bicycle purchased.

PART 2

Depending on the data type of each attribute, transform each object from your preprocessed data. Use all the data rows (= 18000 rows) with the selected features as input to apply all the tasks below, do not perform each task on the smaller data set that you got from your random sampling result.

- (a) Handling Null values
- (b) Normalization
- (c) Discretization (Binning) on Continuous attributes or Categorical Attributes with too many different values
- (d) Standardization/Normalization
- (e) Binarization (One Hot Encoding)

SOLUTION

(a) Handling Null Values

Checking for Null values in the DataFrame:

```
print(df_selected.isnull().sum())
```

OUTPUT

```
Age          0
Education    0
Occupation   0
Gender       0
MaritalStatus 0
HomeOwnerFlag 0
NumberCarsOwned 0
YearlyIncome 0
dtype: int64
```

We have no null values in the selected features, so no action is needed.

However, if there were null values, we could handle them by either dropping the rows or filling them with appropriate values (imputation). This can be done as follows:

- (a) Dropping rows with null values:

```
df_selected.dropna(inplace=True)
```

- (b) Filling null values with the mean (for continuous variables) or mode (for categorical variables):

```
for col in df_selected.columns:
    if df_selected[col].dtype == 'object': # Categorical
        df_selected[col].fillna(df_selected[col].mode()[0], inplace=True)
    else: # Continuous
        df_selected[col].fillna(df_selected[col].mean(), inplace=True)
```

- (b) Normalization

```
from sklearn.preprocessing import MinMaxScaler
```

```
numeric_cols = df_selected.select_dtypes(include=[np.number]).columns
scaler = MinMaxScaler()
df_selected[numeric_cols] = scaler.fit_transform(df_selected[numeric_cols])
print(df_selected.head())
```

OUTPUT

	Age	HomeOwnerFlag	NumberCarsOwned	YearlyIncome
0	0.185714	1.0	0.6	0.496842
1	0.400000	1.0	0.4	0.489453
2	0.214286	0.0	0.6	0.536172
3	0.328571	1.0	0.4	0.317083
4	0.357143	1.0	0.2	0.231958

(c) Discretization (Binning) on Continuous attributes or Categorical Attributes with too many different values

```
df_selected['Age_binned'] = pd.cut(df_selected['Age'], bins=4, labels=False)
```

OUTPUT

	Age_binned
0	9555
1	7208
2	1544
3	54

Name: count, dtype: int64

(d) Standardization/Normalization

```
std_scaler = StandardScaler()
df_selected[numeric_cols] = std_scaler.fit_transform(df_selected[numeric_cols])
```

OUTPUT

	Age	HomeOwnerFlag	NumberCarsOwned	YearlyIncome
0	-0.482516	0.798603	1.892524	0.298555
1	0.851033	0.798603	0.798389	0.271180
2	-0.304710	-1.252187	1.892524	0.444261
3	0.406517	0.798603	0.798389	-0.367401
4	0.584324	0.798603	-0.295746	-0.682765

(e) Binarization (One Hot Encoding)

```
categorical_cols = df_selected.select_dtypes(exclude=[np.number]).columns
encoder = OneHotEncoder(sparse_output=False)
encoded_data = encoder.fit_transform(df_selected[categorical_cols])
encoded_df = pd.DataFrame(encoded_data,
                           columns=encoder.get_feature_names_out(categorical_cols))
df_final = pd.concat([df_selected.drop(columns=categorical_cols), encoded_df],
                     axis=1)
```

```
print(df_final.columns)
```

OUTPUT

```
Index(['Age', 'HomeOwnerFlag', 'NumberCarsOwned', 'YearlyIncome', 'Age_binned',
      'Education_Bachelors', 'Education_Graduate Degree',
      'Education_High School', 'Education_Partial College',
      'Education_Partial High School', 'Occupation_Clerical',
      'Occupation_Management', 'Occupation_Manual', 'Occupation_Professional',
      'Occupation_Skilled Manual', 'Gender_F', 'Gender_M', 'MaritalStatus_M',
      'MaritalStatus_S'],
      dtype='object')
```

PART 3

Make sure each attribute is transformed in a same scale for numeric attributes and Binarization for each nominal attribute, and each discretized numeric attribute to standardization. Make sure to apply a correct similarity measure for nominal (one hot encoding)/binary attributes and numeric attributes respectively.

- (a) Calculate Similarity in Simple Matching, Jaccard Similarity, and Cosine Similarity between two following objects of your transformed input data.
- (b) Calculate Correlation between two features Commute Distance and Yearly Income

SOLUTION

- (a) **Calculating Similarity Measures between the first two columns of the transformed DataFrame:**

```
obj1 = df_final.iloc[0].values.reshape(1, -1)
obj2 = df_final.iloc[1].values.reshape(1, -1)

smc = (obj1 == obj2).sum() / len(obj1[0])
cos_sim = cosine_similarity(obj1, obj2)[0][0]
jac_sim = 1 - jaccard(encoded_df.iloc[0], encoded_df.iloc[1])

print(f"Simple Matching Coefficient: {smc}")
print(f"Cosine Similarity: {cos_sim}")
print(f"Jaccard Similarity: {jac_sim}")
```

OUTPUT

```
Cosine Similarity: 0.6200078998918396
Jaccard Similarity: 0.6
Simple Matching Coefficient: 0.6842105263157895
```

- (b) **Calculating Correlation between 'Commute Distance' and 'Yearly Income':**

```
corr = np.corrcoef(df['NumberCarsOwned'], df['YearlyIncome'])[0, 1]
print(f"Correlation (NumberCarsOwned vs YearlyIncome): {corr}")
```

OUTPUT

```
Simple Matching Coefficient: 0.6842105263157895
Correlation (NumberCarsOwned vs YearlyIncome): 0.47730015236316964
```